REPUBLIC OF THE PHILIPPINES

UNIVERSITY OF CABUYAO

(Pamantasan ng Cabuyao) | College of Computing Studies

Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

PANADERO

BAKING SYSTEM

System Overview • Requirements • Features • Database • SQL Demonstration • Implementation

DOCUMENT PURPOSE

This document describes the PANADERO local bakery ordering and management system. It records the application's roles and workflows, database design, example SQL operations, setup approach, and implementation constraints for review and maintenance.

Prepared 12 October 2026 | Version 1.0 | Based on the supplied PANADEROFINAL project archive

Local/classroom edition — not yet validated against a running MySQL instance

Contents

- 1. System Overview
- 2. Functional Requirements
- 3. Non-Functional Requirements
- 4. System Features and Screenshots
- 5. Database Design and ERD
- 6. SQL Query Demonstration
- 7. System Implementation and Setup
- 8. Assignment Readiness and Known Gaps
- 9. Security, Reliability, and Test Notes
- 10. References and File Map

Reading note: portal screenshots in Section 4 are UI previews generated from the supplied front-end with an isolated, temporary fixture server. Their example names, quantities, dates, and totals are illustrative only; they are not live bakery records and do not constitute an end-to-end database test.

1. System Overview

1.1 System title

PANADERO Baking System — a local web application for bakery product browsing, ordering, reservations, inventory batches, and request administration.

1.2 Problem statement

A small bakery can receive customer orders and advance reservations while also tracking product availability, fresh production batches, and request status. If catalog, request, and stock information is handled across separate paper notes or informal messages, staff can lose track of availability, repeat data entry, or provide inconsistent status updates. The system addresses this coordination problem by putting the customer, staff, and administrator workflows behind one application and one relational database.

1.3 Solution and system boundary

PANADERO provides three role-oriented browser portals. A Node.js/Express API mediates between browser pages and a MySQL/MariaDB database. Customers browse a catalog, maintain a browser cart, submit orders or reservations, and view their own request history. Staff record fresh batches, adjust stock, and move customer requests through statuses. Administrators maintain the catalog, categories, users, stock, and request overview. Bakery product images are static assets; the API and database contain operational data.

1.4 Target users

User	Primary activities
Customer	Browse and search products; register/sign in; place an order; request a scheduled reservation; review order/reservation status and notifications.
Bakery staff	Review operational dashboard; record baked batches and stock; find customer orders/reservations; advance request statuses.
Administrator	Oversee system state; maintain categories/products/users/stock; review customer requests and operational summaries.
Project maintainer	Install dependencies, configure MySQL, run the local

User	Primary activities
	server, update schema/source, and validate the system.

1.5 System objectives

- Present bakery products and current availability through a customer-facing catalog.
- Accept customer orders and reservations and return a reference for tracking.
- Calculate order totals and update stock using server-side data and database transactions.
- Give staff a central view of stock, production batches, and customer requests.
- Let authorized administrators maintain shared catalog and account records.
- Keep application code, SQL schema, setup instructions, and technical documentation in one maintainable project.

1.6 Architecture at a glance

```
Customer portal ─┐
Staff portal    ─┼── HTTP/JSON ── Node.js + Express API ── mysql2 ── MySQL/MariaDB
Admin portal    ─┘
Static HTML/CSS/JavaScript and images are served by the same local Express application.
```

The browser does not connect directly to the SQL database. The backend owns database credentials and applies role checks, validation, price/stock lookups, and database writes.

2. Functional Requirements

The following requirements describe the visible system behavior documented in the supplied source. “Expected output” means the response or screen state on a successful request unless an error condition is stated.

ID / function	Description	Authorized user	Expected output
FR-01 Catalog browsing	The system shall display bakery categories, products, prices, images, and availability.	Visitor / Customer	Catalog cards and category/product data returned by GET /api/catalog.
FR-02 Customer account	The system shall support customer registration, sign-in, sign-out, and password-reset request capture.	Customer / Visitor	Account/session response; invalid credentials return an error; reset endpoint returns a generic request message.
FR-03 Cart and checkout	The system shall let a customer review selected product quantities and submit an order.	Customer	Order confirmation and reference such as ORD-00001; backend calculates total from current SQL prices.
FR-04 Order records	The system shall create an order header and line items and decrease stock atomically.	Customer	An orders record, linked order_items records, and updated bread_products stock; rollback on transaction failure.
FR-05 Reservations	The system shall accept a product quantity and preferred date/time for a reservation.	Customer	Reservation reference such as RES-00001 with a pending status and scheduled details.
FR-06 Customer activity	The system shall return a signed-in customer’s orders, reservations, and completion notifications.	Customer	Only the authenticated user’s request history and their notification read/unread state.
FR-07 Staff dashboard	The system shall summarize recorded stock, pending requests, ready requests, and stock needing attention.	Staff	Metric cards and a recent order/reservation list.
FR-08 Inventory and batches	The system shall let staff set nonnegative whole-number stock and record baked quantity/date/expiry/notes.	Staff	Updated stock quantity and an inventory_batches record; a new batch increases product stock.
FR-09 Request status workflow	The system shall let staff advance order and reservation status along the allowed	Staff; Admin cancellation where permitted	Updated request status; invalid/backward transitions are rejected.

ID / function	Description	Authorized user	Expected output
	workflow.		
FR-10 Administration	The system shall expose management views for categories, products, users, inventory, orders, reservations, and operational summaries.	Administrator	Admin state view and persisted category/product/user/stock changes; request status changes use validated routes.
FR-11 Health/build status	The system shall expose build information and database health endpoints for setup checks.	Maintainer / Operations	Build identifier and a health response; when the database is unavailable the health endpoint reports service unavailable.

3. Non-Functional Requirements

Quality attribute	How the current system addresses it / verification limit
Usability	Separate customer, staff, and admin navigation with task-focused pages, search/category filters, visible statuses, and responsive CSS. A real-user usability study was not supplied.
Performance	The application serves static assets and makes focused JSON API calls; product, order, and inventory data are loaded from MySQL through mysql2. No load test or response-time benchmark was run; performance under concurrent or production load is unverified.
Reliability	The schema uses InnoDB and foreign keys. Order creation and fresh-batch recording use transactions so related record/stock changes commit or roll back together. Startup includes compatibility table creation and initial seeding behavior. Database-backed reliability still depends on a configured, running MySQL service.
Security	Passwords are hashed with bcryptjs; protected endpoints use bearer-token authentication and role middleware; SQL statements use parameter placeholders in key API operations; customer history is scoped to the signed-in user. Current limitations include in-memory sessions, broad default CORS, no documented rate limiting/HTTPS/CSRF deployment layer, and demo accounts; harden before public deployment.
Maintainability	Frontend assets are grouped by role; API/data access is centralized in server.js and service scripts; schema and setup notes are in database/ and docs/; npm scripts provide repeatable start/dev commands. Several older nested folder names are inconsistent (including “COSTUMER”) but must be preserved unless paths are refactored together.

4. System Features and Screenshots

This section describes each primary feature/module, how it is used, the principal data it processes, and its user-facing result. Preview screenshots use the application’s real HTML/CSS/JavaScript layouts with the temporary fixture server noted above.

4.1 Customer portal — home and catalog

The storefront introduces the bakery and routes visitors to the menu. The menu displays products under categories such as Breads, Cakes, Doughnuts, and Pies, with product names, prices, images, category filters, and search. A customer selects a product to view its details or add it to the browser cart.

Data processed: category ID/name/description; product ID/name/category/price/image and current stock; local cart product IDs and quantities. Output: browsable product cards, category results, availability cues, and updated cart count.

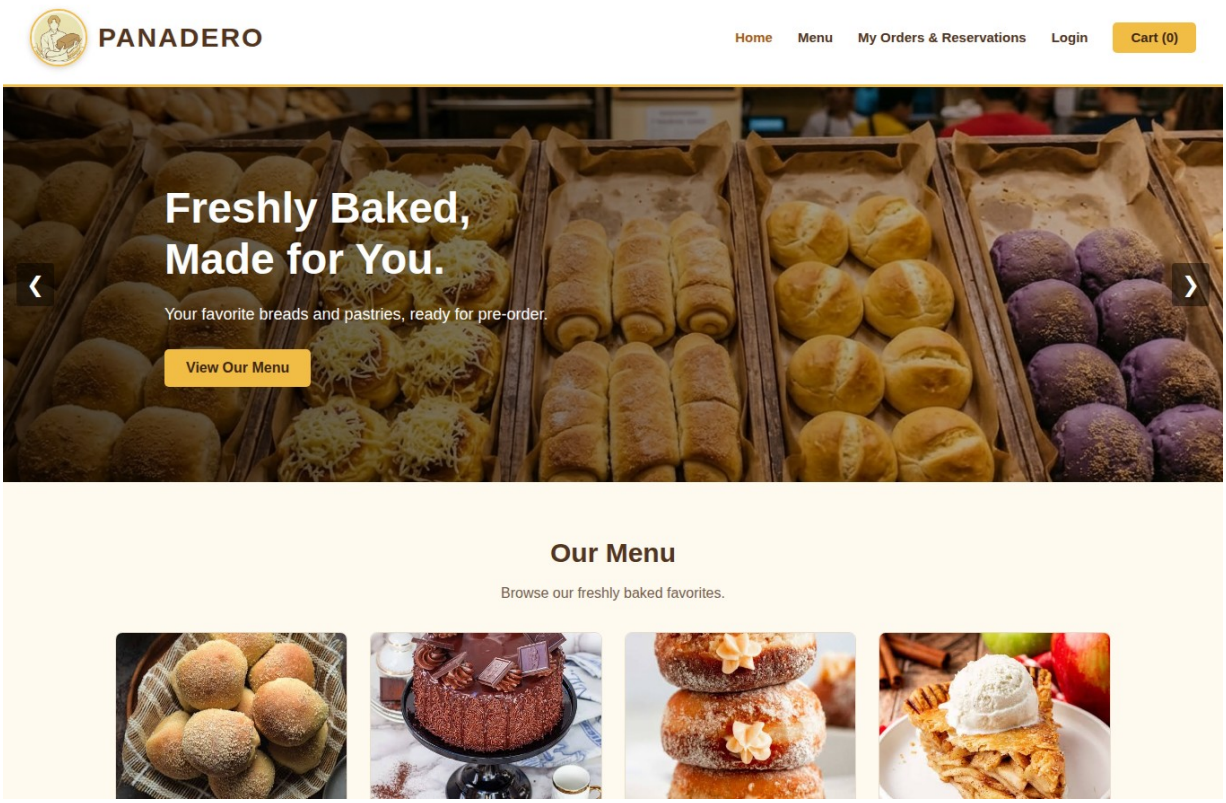


Figure 1. Customer storefront home page (frontend preview).

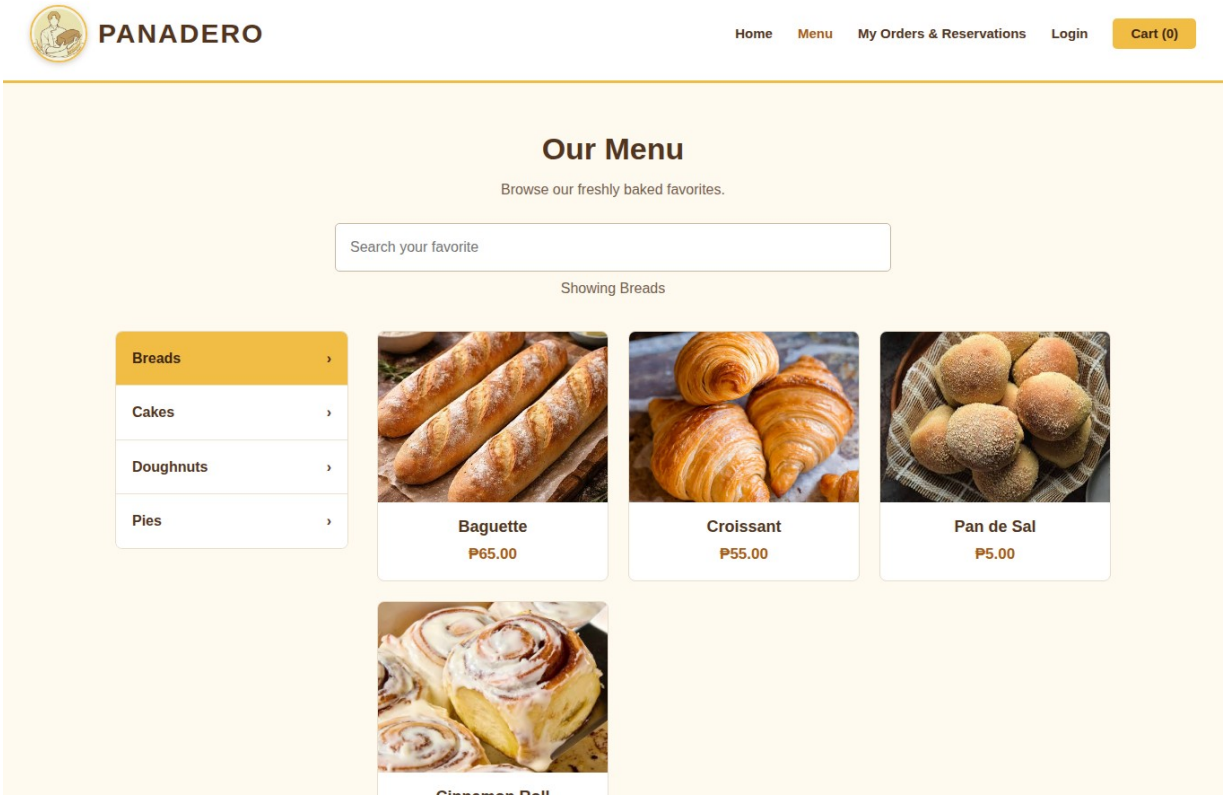


Figure 2. Product menu with category filters and product cards (frontend preview).

4.2 Customer authentication, order, and tracking

Registration/sign-in submits account details to the API; successful sign-in creates a bearer-token session. Checkout sends product IDs and quantities rather than trusting a client-side total. The API reloads current price/stock, validates the request, writes an order and order_items, and returns an order reference. The reservation flow records a preferred date/time and the customer's chosen product/quantity. The tracking page loads only the signed-in customer's orders/reservations; completion notices can be listed and marked read.

Data processed: account name/email/password (password stored as a hash), cart lines, current product price/stock, request date/time, status, and notification state. Output: authenticated session, confirmation/reference, request history/status, or validation/error message. Payment gateways and actual card processing are not part of the supplied implementation.

4.3 Staff dashboard

The dashboard summarizes available stock, pending and ready customer requests, and products with stock needing attention. A recent-requests table supports quick review before staff opens the full active-requests page.

Data processed: inventory quantities, request type/reference, customer display name, total, status, and submitted date. Output: operational metrics and a recent request list.

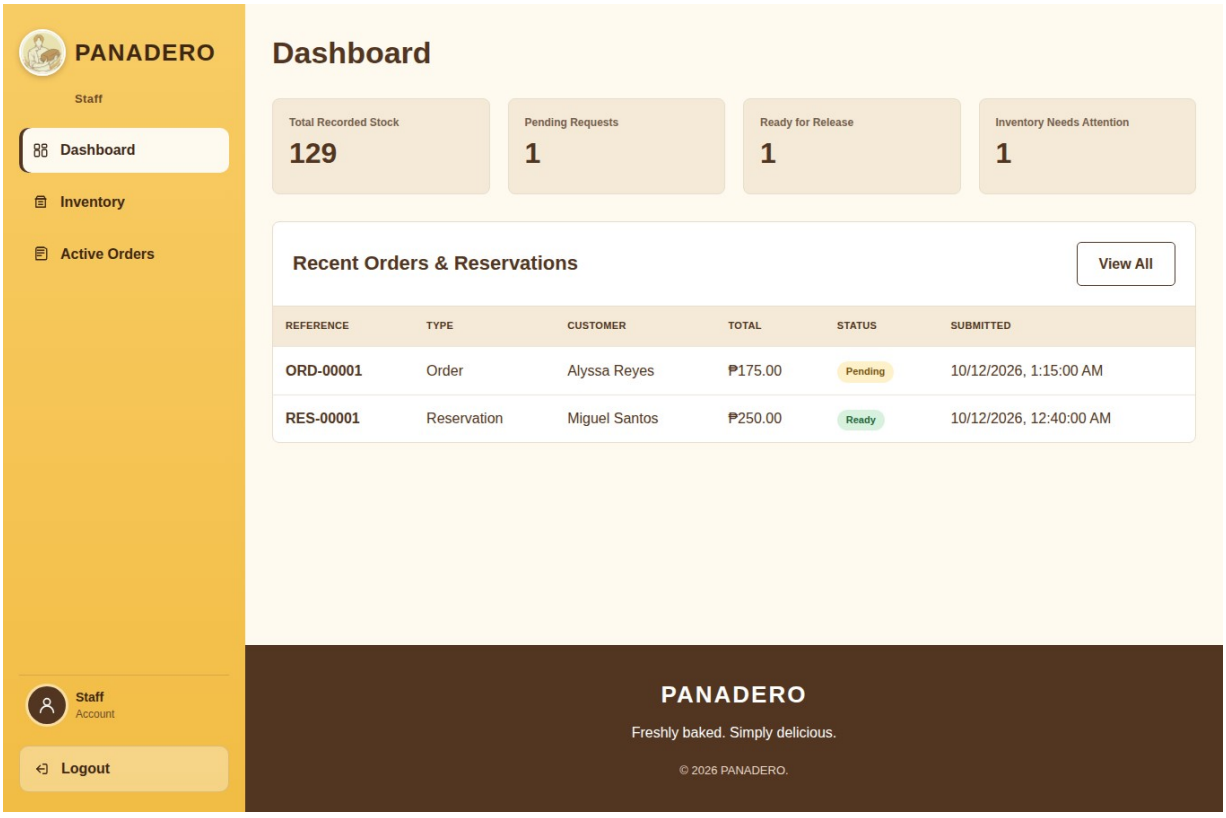


Figure 3. Staff dashboard; names and request rows are illustrative fixture data.

4.4 Inventory and fresh-batch management

Staff can search/filter the product inventory, save a nonnegative whole-number stock value, and create a fresh-batch record with product, quantity, baked date, expiry date, and notes. Batch creation increases the product stock within a transaction. The screen shows availability and recent baking batches.

Data processed: product/category/price/current stock, batch quantity, baked/expiry dates, and notes. Output: updated stock, batch history, and in-stock/out-of-stock states.

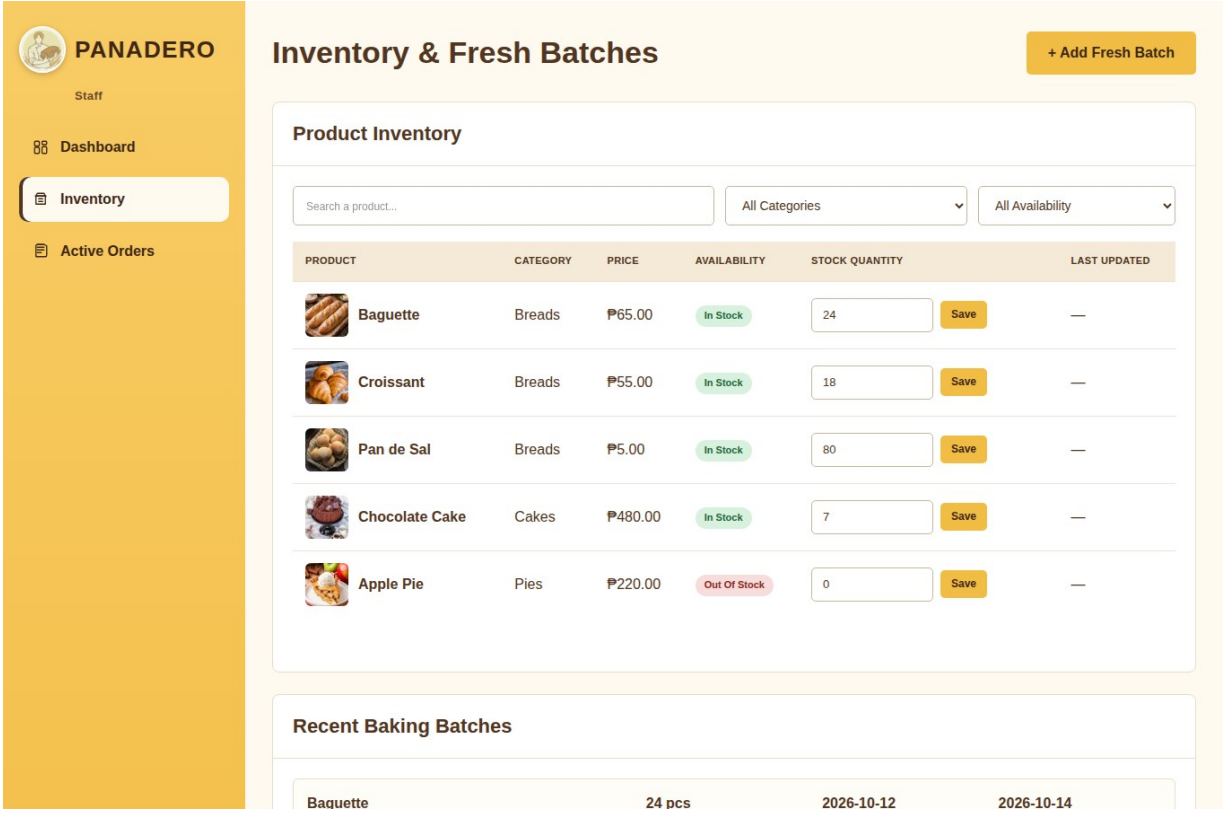


Figure 4. Staff inventory and fresh-batch screen; quantities are illustrative fixture data.

4.5 Staff order and reservation workflow

The active-requests page consolidates orders and reservations. Staff may search references/customers/products, filter by request type or status, and advance a request to the next valid state. The documented progression is Pending → Preparing/Processing → Ready → Completed; cancellation is constrained by the API rules.

Data processed: reference, customer contact, ordered/reserved products and quantities, fulfillment/schedule details, total, and status. Output: searchable request rows and a validated saved status.

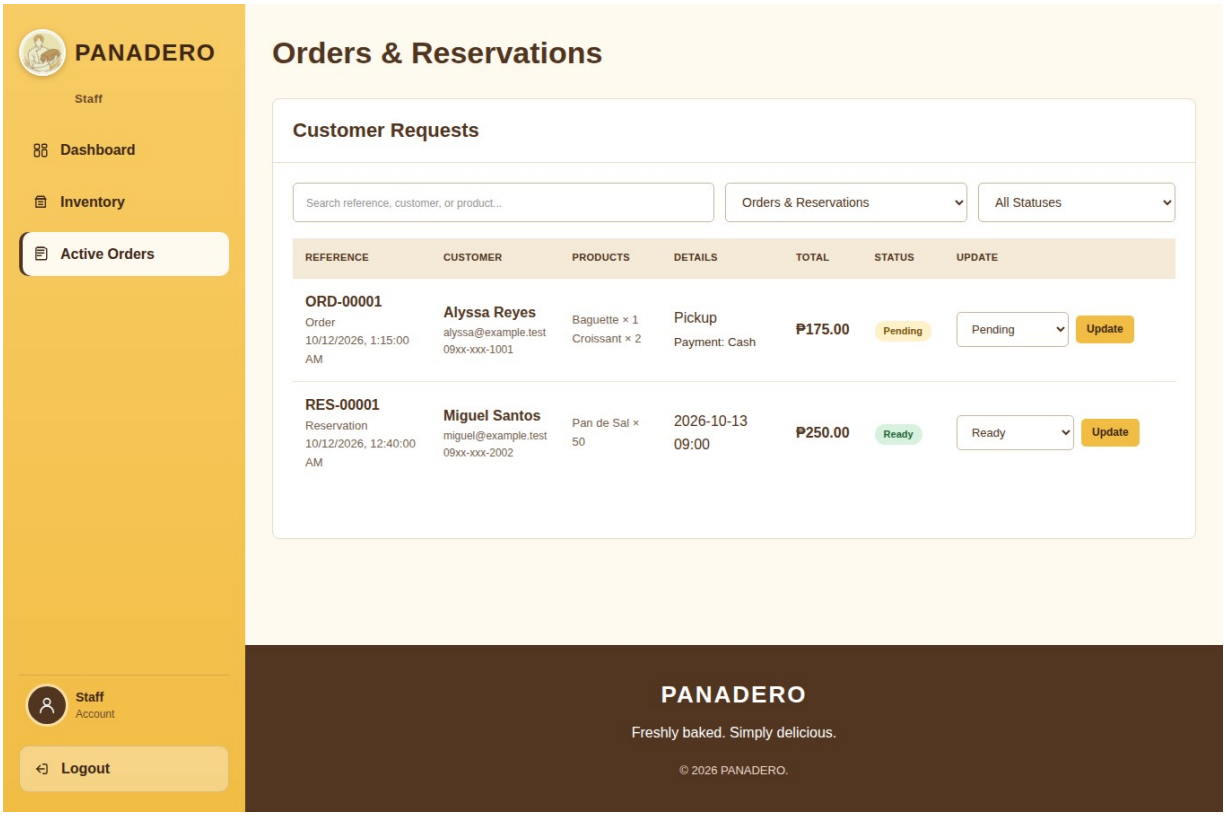


Figure 5. Staff order/reservation queue; all displayed example records are fixture data.

4.6 Administrator portal

The administrator dashboard summarizes stock/request state and provides navigation to inventory/batches, orders/reservations, customers, categories, users, and sales/log views. The API state/synchronization routes support persisted category, product, user, and stock edits; request status changes are handled by the validated status routes.

Data processed: shared catalog/category data, account role/display data, stock/batches, orders/reservations, and summary counts. Output: admin overview, management views, and saved changes. The system_logs table is present, but the current admin state returns an empty log list; full audit-log writes remain future work.

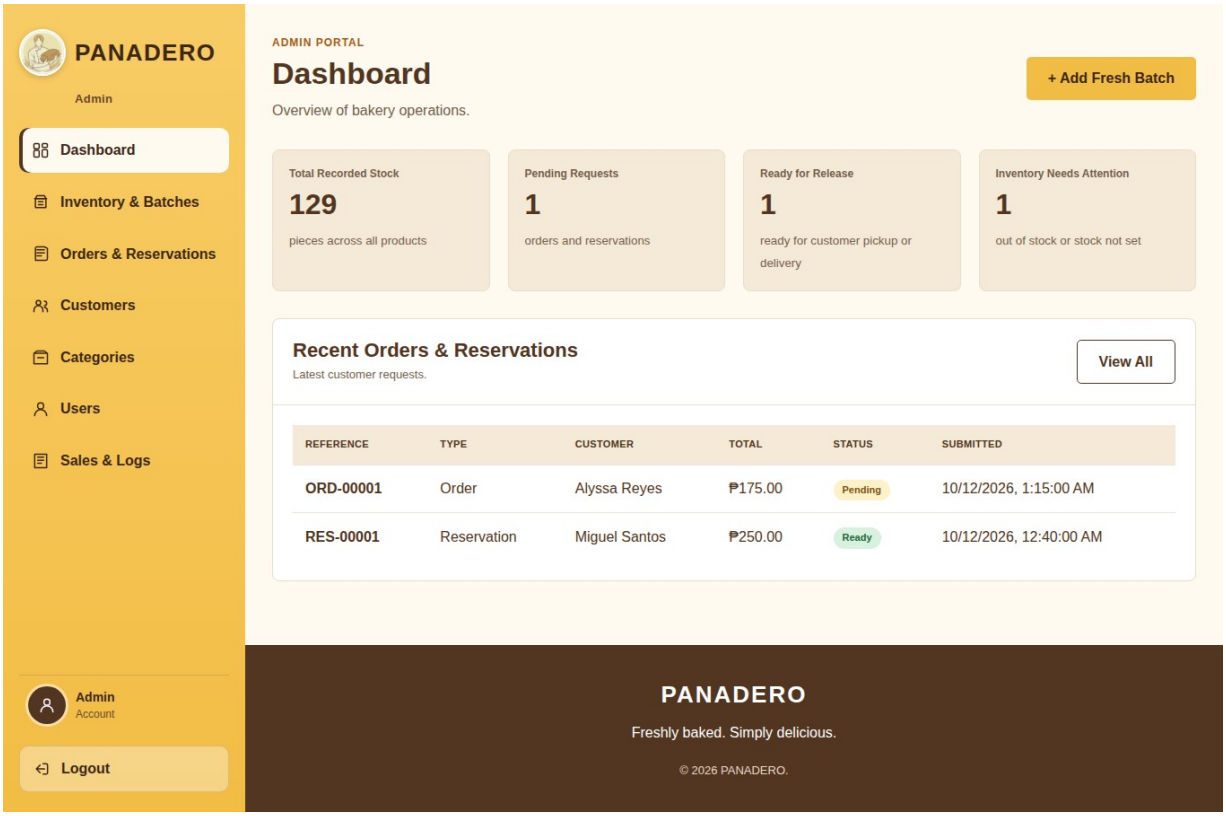


Figure 6. Administrator dashboard; example totals and names are illustrative fixture data.

5. Database Design and ERD

The canonical SQL creates the panadero_bakery database and nine related tables. The design has more than the required six tables, three main entity tables (users, categories, bread_products), transaction tables (orders/reservations), and a junction table (order_items) for the many-to-many relationship between orders and products.

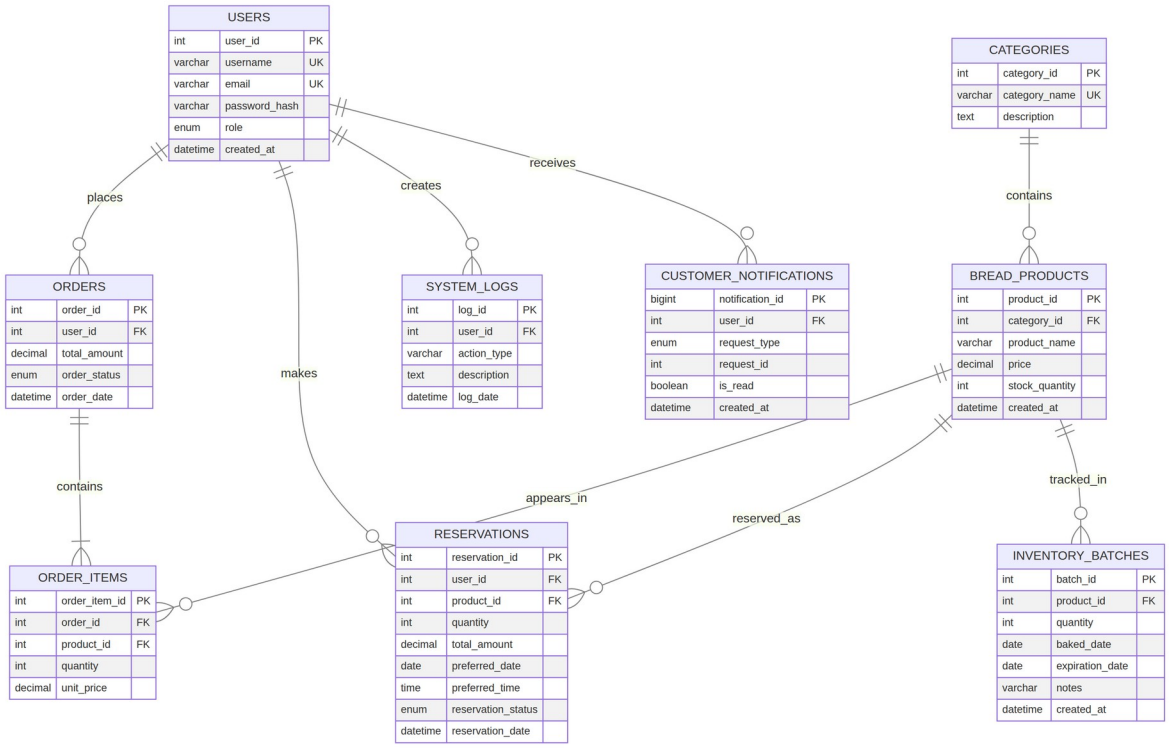


Figure 7. PANADERO ERD rendered from the supplied SQL and schema notes.

5.1 Table catalogue

Table	Primary/foreign keys	Purpose and representative data
users	user_id PK; username and email unique	Customer, Staff, and Admin accounts; password_hash stores a bcrypt hash.
categories	category_id PK; category_name unique	Product groups such as breads, cakes, doughnuts, and pies.
bread_products	product_id PK; category_id FK → categories	Bakery item name, category, decimal price, current stock, and created date.
orders	order_id PK; user_id FK → users	Customer order header, total, status, and order date.
order_items	order_item_id PK; order_id FK → orders; product_id FK → bread_products	Order/product bridge; quantity and unit_price preserve the price used for the order line.
reservations	reservation_id PK; user_id FK → users; product_id FK → bread_products	Scheduled product reservation, quantity, amount, preferred date/time, and status.
inventory_batches	batch_id PK; product_id FK → bread_products	Production batch quantity, baked/expiration dates, notes, and record timestamp.
system_logs	log_id PK; nullable user_id FK → users	Audit-log structure for action type, description, and time; current API does not write/populate these rows.
customer_notifications	notification_id PK; user_id FK → users; unique user/type/request key	Per-customer order/reservation completion notification, read state, and timestamp.

5.2 Relationships and constraints

- One user may have many orders, reservations, system-log records, and customer notifications.
- One category contains many bread_products; one product may appear in many order_items, reservations, and inventory_batches.
- Orders and products form a many-to-many relationship resolved by order_items; the bridge stores quantity and historical unit price.

- The SQL declares foreign keys with explicit delete/update actions. Order deletion cascades to its order_items; product/category/user deletion is generally restricted where operational records depend on the row; logs can retain a null actor after user deletion.
- Current stock is stored on bread_products.stock_quantity. Reservation and order creation reduce stock; receiving a batch increases it. These changes are implemented in server-side workflows.

5.3 Schema source and seed-data note

Canonical import: database/panadero.sql. The server also contains safe CREATE TABLE IF NOT EXISTS compatibility setup and seeds categories, products, and demo Staff/Admin accounts when tables are empty. The SQL file itself primarily defines the schema and does not contain a complete sample INSERT set. The runtime seeder creates four categories and two demo staff/admin accounts; therefore it does not establish five sample user rows, and the project should be augmented with a documented, repeatable test-data seed to meet the assignment's "minimum five records per main table" requirement. Do not create real customer identities merely to satisfy a sample-count checklist.

6. SQL Query Demonstration

The following examples are intended for a disposable test copy of panadero_bakery. They are shown but were not executed in this workspace because MySQL/MariaDB was unavailable. Run them only against a test database; the temporary category/product rows should be removed afterward.

6.1 SELECT — view the catalog

```
SELECT p.product_id, p.product_name, c.category_name, p.price, p.stock_quantity
FROM bread_products AS p
JOIN categories AS c ON c.category_id = p.category_id
ORDER BY c.category_name, p.product_name;
```

Purpose: combine products with their category names and display a stable catalog ordering. Expected output: one row per product, including category, current price, and stock.

6.2 INSERT — create a temporary demo category and product

```
INSERT INTO categories (category_name, description)
VALUES ('SQL Demo Category', 'Temporary row for the class SQL demonstration');

INSERT INTO bread_products (category_id, product_name, price, stock_quantity)
SELECT category_id, 'SQL Demo Loaf', 99.00, 5
FROM categories WHERE category_name = 'SQL Demo Category';
```

Purpose: add two related test rows and demonstrate a category foreign-key relationship. Expected output: one category and one linked product are inserted, assuming the demo category does not already exist.

6.3 UPDATE — change the temporary product price

```
UPDATE bread_products
SET price = 105.00
WHERE product_name = 'SQL Demo Loaf'
  AND category_id = (SELECT category_id FROM categories
                     WHERE category_name = 'SQL Demo Category');
```

Purpose: change only the temporary demo product. Expected output: one product row has the revised price; verify with SELECT and inspect affected-row count.

6.4 DELETE — remove the temporary demo product and category

```
DELETE FROM bread_products
WHERE product_name = 'SQL Demo Loaf'
  AND category_id = (SELECT category_id FROM categories
```

```
                    WHERE category_name = 'SQL Demo Category');

DELETE FROM categories
WHERE category_name = 'SQL Demo Category';
```

Purpose: remove only the test records, deleting the child product before its category to respect the foreign key. Expected output: both demo rows are removed; no production catalog rows are affected.

6.5 Advanced query — request totals by status and product category

```
SELECT o.order_status, c.category_name, COUNT(DISTINCT o.order_id) AS order_count,
       SUM(oi.quantity * oi.unit_price) AS item_sales
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN bread_products AS p ON p.product_id = oi.product_id
JOIN categories AS c ON c.category_id = p.category_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL 30 DAY
GROUP BY o.order_status, c.category_name
ORDER BY item_sales DESC, o.order_status;
```

Purpose: illustrate JOIN + WHERE + GROUP BY + ORDER BY for a last-30-days operational summary. Expected output: one row per order-status/category pair that has matching order lines, with order count and line-amount total. This is a demonstration query; it is not presented as a built-in analytics screen.

7. System Implementation and Setup

7.1 Technology stack

Layer	Technology	Responsibility
Presentation	HTML5, CSS3, browser JavaScript	Role-specific portal pages, forms, filters, cart, and rendering.
Application/API	Node.js 18+ and Express 4	JSON routes, validation, authorization middleware, business logic, static file serving.
Database access	mysql2	MySQL connection pool and parameterized queries.
Data store	MySQL 8 or MariaDB	Relational tables, constraints, durable request and catalog data.
Security/config	bcryptjs, dotenv, CORS	Password hash/verify, local configuration, cross-origin support (production origin should be restricted).
Local launch	Windows Batch launcher plus npm scripts	Database import and local setup assistance; `npm start` or `npm run dev`.

7.2 Project map

```
PANADERO/
├── server.js                       Express API and static server
├── package.json / package-lock.json  Runtime dependencies and npm commands
├── .env.example                    Safe local configuration template
├── database/panadero.sql           Canonical MySQL schema
├── PANADERO-COSTUMER/...           Customer UI, CSS, JavaScript, images
├── PANADERO-STAFF/...              Staff dashboard, stock, request pages
├── PANADERO-ADMIN/...              Administrator console and assets
├── docs/                            ERD, guides, screenshots, this document
└── PANADERO-ONE-CLICK.bat           Windows startup helper
```

7.3 Local setup

1. Install Node.js 18+ and MySQL 8 or MariaDB; start the database service.
2. Import the schema: mysql -u root -p < database/panadero.sql (use appropriate local credentials).

3. Copy .env.example to .env and set DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME and PORT. Keep .env private and out of Git.
4. Run npm install, then npm start from the project root. The default application port is 3000.
5. Open http://localhost:3000/customer/login.html. Staff/admin operations require the role-appropriate account and API authorization.
6. On Windows, the included PANADERO-ONE-CLICK.bat provides the project's assisted local startup path; see LOCAL-SETUP.md for requirements and troubleshooting.

7.4 Key API route groups

Route group	Representative endpoints	Purpose
Authentication	POST /api/auth/register, /login, /logout, /password-reset	Create/validate customer session and record reset request.
Catalog	GET /api/catalog	Read categories and products.
Customer requests	POST /api/orders, /api/reservations; GET /api/orders, /api/reservations	Create requests and retrieve signed-in customer history.
Staff	GET /api/inventory, /api/staff/requests; PATCH stock/status; POST batches	Manage stock, batch records, and request progress.
Admin	GET /api/admin/state; POST /api/admin/sync	Hydrate and persist supported administrative changes.
Notifications / health	GET /api/notifications; PATCH notification read; GET /api/health, /api/build	Read/mark notices and confirm build/database availability.

8. Assignment Readiness and Known Gaps

This status is based on inspection of the supplied files and brief, not on a database-backed acceptance run.

Brief criterion	Current evidence	Status
System overview	Problem, solution, target users, objectives, architecture documented above.	Covered
Functional requirements	Eleven requirements state behavior, user access, and expected output.	Covered
Non-functional requirements	Usability, performance, reliability, security, and maintainability with caveats.	Covered
Feature/module explanations and screenshots	Customer, Staff, and Admin workflows described with six UI previews; ERD figure included.	Covered with fixture-data note
At least six related tables	Nine tables in the canonical SQL schema.	Meets
At least three main entities	users, categories, bread_products.	Meets
Transaction/log table	orders, reservations, inventory_batches, and system_logs are present.	Meets structurally
Many-to-many bridge	order_items links orders and bread_products and stores quantity/unit_price.	Meets
Primary/foreign keys	All tables have a primary key; multiple explicit foreign-key relations are declared.	Meets structurally
At least five sample rows per main table	SQL file does not provide the full sample seed; server seeds four categories and two demo accounts.	Needs work
CREATE DATABASE / CREATE TABLE / sample INSERT script	CREATE statements exist; repeatable >=5-record seed INSERTs are not in the canonical SQL file.	Needs work
CRUD + advanced SQL demonstration	Document includes SELECT, INSERT, UPDATE, DELETE, and an advanced joined aggregate query.	Documented; not run
Working application and API	Source and setup instructions are present; live MySQL/API integration was not validated in this environment.	Needs local test

Recommended completion items

- Add idempotent, safe, clearly labeled sample seed SQL that produces at least five appropriate records for each instructor-designated main table; avoid weak or reusable sample passwords.
- Start MySQL/MariaDB locally, import the canonical script, run npm start, and verify /api/health before demonstrating the flows.
- Execute CRUD/advanced queries against a disposable database and record observed outputs for the class demonstration.
- Capture final screenshots from the configured application using the instructor-approved data state, replacing illustrative fixture screenshots if required.
- Before any public deployment, restrict CORS, require HTTPS, add rate limiting and stronger session lifecycle controls, remove demo credentials, and verify staff/admin endpoint authorization.

9. Security, Reliability, and Test Notes

9.1 Observed safeguards

- Password values are hashed with bcryptjs before database storage; plaintext passwords are not intended to be persisted.
- Role middleware protects staff/admin API route groups; customer history queries are scoped using the authenticated user ID.
- The order workflow reloads price and stock from the database rather than trusting browser totals and uses a transaction to keep order lines and stock updates consistent.
- The SQL schema uses InnoDB, primary keys, uniqueness constraints, and foreign keys to preserve related records.
- The root .gitignore excludes node_modules, .env, and database_backups/; the supplied SQL backup contains account/email/password-hash fields and should not be uploaded as a repository fixture.

9.2 Limitations and operational cautions

- This application is described as a local/classroom system. Its session tokens are held in the running server process and are not durable across server restarts.
- The application’s default CORS middleware is broad. Configure allowed origins for the actual deployment environment.
- The password-reset route records/responds to a reset request but does not itself send a verified recovery email.
- The system_logs schema exists, but the current admin state supplies an empty logs list; it should not be represented as a fully functioning audit trail.
- The app supports request totals and a payment-method field in the UI, but no payment gateway or card-processing integration was found in the supplied project.
- The screenshots in this document use a temporary fixture server with sample rows. No fixture server, preview auth page, API token, or sample business data is part of the application release.
- The supplied database_backups/ snapshot contains account identifiers and password-hash fields. It is excluded from Git by the updated ignore rule; only the canonical schema and sanitized, repeatable seed data should be published.

9.3 Validation performed for this document

The source archive, README, setup guides, canonical SQL, API route declarations, role pages, and existing ERD notes were inspected. Portal screenshots were rendered from the supplied frontend using

isolated temporary fixture responses. A direct startup attempt confirmed this workspace had no MySQL service listening on 127.0.0.1:3306; consequently, database health, schema import, application flows, CRUD queries, data persistence, and performance were not tested end-to-end.

10. References and File Map

File	Use
README.md	Project overview, role portals, quick start, and technical summary.
LOCAL-SETUP.md	Prerequisites, database import, local configuration, portal URLs, troubleshooting.
server.js	API endpoints, request logic, role checks, migrations, seed behavior, and static serving.
database/panadero.sql	Canonical database/schema definition for panadero_bakery.
docs/PANADERO-LOCAL-ERD-UPDATED.md	Existing ERD and relationship interpretation notes.
docs/PANADERO-ERD.mmd / .png	ERD source/image embedded in this report, including all nine tables.
docs/PANADERO-SYSTEM-GUIDE.md	Existing architecture and workflow explanation.
docs/screenshots/	Portal preview screenshots included in this report; images contain illustrative fixture records.
docs/PANADERO-System-Documentation.docx	This consolidated Word deliverable.
docs/PANADERO-System-Documentation.pdf	PDF companion copy of the Word deliverable.

Source basis: PANADEROFINAL.zip and the accompanying course requirements note supplied with the project. This report describes the local edition and records unverified items as such; it does not certify production security, complete test coverage, or an operational database instance.